

UD05.01: Fundamentos de la POO en Python

Contenido

1. Introducción a la programación orientada a objetos	2
2. Clases y objetos	2
2.1 ¿Qué es una clase?.....	2
2.2 ¿Qué es un objeto?	2
3. Definición de clases e instanciación de objetos	3
4. Atributos y métodos.....	3
4.1 Métodos	3
4.1.1 El parámetro self.....	3
4.2 Métodos con parámetros	4
4.3 Método constructor __init__	4
4.4 Atributos.....	4
5. Principios básicos de la POO	6
5.1 Encapsulamiento	6
5.2 Herencia.....	7
5.3 Polimorfismo	8
5.4 Abstracción	9

1. Introducción a la programación orientada a objetos

La programación orientada a objetos (POO) es un paradigma que organiza el código en **clases** y **objetos**, facilitando la reutilización, el mantenimiento y la organización del software.

Python es un lenguaje completamente orientado a objetos: **todo es un objeto**, incluidos enteros, cadenas, listas, etc.

Ejemplo:

```
texto = "Hola mundo"
```

```
print(texto.upper()) # upper() es un método del objeto texto
```

En este caso, el objeto texto pertenece a la clase str, y puede utilizar métodos como upper(), lower(), replace(), etc.

2. Clases y objetos

2.1 ¿Qué es una clase?

Una **clase** es un modelo que define las propiedades (atributos) y comportamientos (métodos) que pueden tener los objetos.

2.2 ¿Qué es un objeto?

Un **objeto** es una **instancia** de una clase, es decir, una copia creada en memoria con sus propios datos.

3. Definición de clases e instanciación de objetos

La creación de una clase en Python se realiza con la palabra clave `class`.

Para crear un objeto, simplemente se llama a la clase como si fuera una función.

Ejemplo_01:

```
class Persona:
```

```
    pass
```

```
p1 = Persona() # Instanciación del objeto p1
```

Aquí `p1` es un objeto de la clase `Persona`.

4. Atributos y métodos

Los **atributos** son variables que pertenecen a un objeto.

Los **métodos** son funciones definidas dentro de la clase que permiten a los objetos realizar acciones.

4.1 Métodos

Los métodos son funciones definidas dentro de la clase que permiten a los objetos realizar acciones.

4.1.1 El parámetro `self`

El primer parámetro que recibe cualquier método de una clase hace referencia al propio objeto con el que se está trabajando. Por convención a ese primer parámetro se le llama ``self`` (que podríamos traducir como yo mismo), pero puede llamarse de cualquier forma.

Ejemplo_02:

```
class Persona:
```

```
    def saludar(self):
```

```
        print("Hola, soy una persona.")
```

```
p = Persona()
p.saludar()
```

4.2 Métodos con parámetros

Los métodos pueden recibir otros parámetros además de self.

Ejemplo_03:

```
class Persona:
    def saludar(self, nombre):
        print(f"Hola, {nombre}")
```

```
p = Persona()
p.saludar("Laura") # Imprime: Hola, Laura
```

4.3 Método constructor __init__

El constructor es un método especial llamado `__init__()` que se ejecuta automáticamente al crear un objeto. Se usa para inicializar atributos.

A continuación se exponen los atributos y un ejemplo válido para ambos.

4.4 Atributos

Los atributos son variables que pertenecen a un objeto.

Los atributos pueden ser de clase o de la instancia.

Se pueden definir dentro del constructor o directamente en la clase. Aunque se definen habitualmente dentro del método constructor `__init__()`.

Ejemplo_04:

```
class Persona:
    at_clase=1    # Atributo de clase

    # Método donde se define un Atributo de instancia
```

```
def metodo(self):  
    self.at_objeto=1    # Atributo de instancia que no se crea hasta que se ejecuta el método.  
    # Método constructor __init__  
def __init__(self, nombre, edad):  
    self.nombre = nombre  # Atributo de instancia que se crea automáticamente con el objeto  
    self.edad = edad  
  
    • self hace referencia al propio objeto.  
    • nombre y edad son atributos únicos para cada instancia de la clase.
```

```
p1 = Persona("Lucía", 30)
```

```
p2 = Persona("Carlos", 25)
```

```
print(p1.nombre) # Lucía
```

```
print(p2.edad) # 25
```

5. Principios básicos de la POO

5.1 Encapsulamiento

El encapsulamiento consiste en ocultar los detalles internos de un objeto y controlar cómo se accede a sus datos.

En Python no existen atributos privados como en otros lenguajes, pero por convención se usa `_atributo` o `__atributo` para indicar que no deben accederse directamente.

Se utiliza para proteger los datos del acceso o modificación indebidos.

Ejemplo_5.1.1:

```
class Producto:
```

```
    def __init__(self, precio):  
        self._precio = precio # Convención de atributo "protegido"
```

Ejemplo_5.1.2:

```
class Cuenta:
```

```
    def __init__(self, saldo):  
        self.__saldo = saldo # Atributo privado
```

```
    def ver_saldo(self):  
        return self.__saldo
```

```
    def ingresar(self, cantidad):  
        if cantidad > 0:  
            self.__saldo += cantidad
```

- El atributo `__saldo` está encapsulado.
- Solo puede accederse a él a través de los métodos `ver_saldo()` e `ingresar()`.

```
c = Cuenta(100)
c.ingresar(50)
print(c.ver_saldo()) # 150
```

Más adelante, se verá cómo usar propiedades para controlar el acceso (ver bloque 2).

5.2 Herencia

Una clase puede heredar los atributos y métodos de otra. Se llama **clase hija** a la que hereda, y **clase base** o **padre** a la que se hereda.

Ejemplo_5.2:

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre

    def saludar(self):
        print(f"Hola, soy {self.nombre}")

# Empleado hereda de Persona
class Empleado(Persona):
    def trabajar(self):
        print(f"{self.nombre} está trabajando")

e1 = Empleado("Ana")
e1.saludar() # Hola, soy Ana
e1.trabajar() # Ana está trabajando
```

5.3 Polimorfismo

El polimorfismo permite que diferentes clases compartan el mismo nombre de método, pero con comportamientos distintos.

Ejemplo_5.3:

```
class Gato:
```

```
    def hablar(self):  
        print("Miau")
```

```
class Perro:
```

```
    def hablar(self):  
        print("Guau")
```

```
# Comportamiento polimórfico:
```

```
def hacer_hablar(animal):  
    animal.hablar()
```

```
hacer_hablar(Gato()) # Miau
```

```
hacer_hablar(Perro()) # Guau
```

5.4 Abstracción

Permite modelar entidades complejas mostrando solo lo necesario. Se define lo esencial y se oculta lo que no es relevante para el uso externo.

Ejemplo_5.4:

```
class Auto:
    def __init__(self):
        self.__motor_encendido = False # detalle interno (oculto al usuario)

    # Métodos públicos (lo que el usuario ve y usa)
    def encender(self):
        self.__activar_bateria()
        self.__inyectar_combustible()
        self.__motor_encendido = True
        print("El auto está encendido")

    def conducir(self):
        if self.__motor_encendido:
            print("El auto se está moviendo")
        else:
            print("No puedes conducir sin encender el auto primero")

    # Métodos privados (detalles internos que el usuario NO necesita conocer)
    def __activar_bateria(self):
        print("(Activando batería...)")

    def __inyectar_combustible(self):
        print("(Inyectando combustible...)")

mi_auto = Auto()

# El usuario solo interactúa con lo esencial:
mi_auto.encender() # No sabe nada de batería o combustible, solo usa "encender"
mi_auto.conducir() # Lógica interna queda oculta
```

Explicación:

El usuario de la clase solo ve dos acciones:

encender()

conducir()

Los detalles internos (__activar_bateria, __inyectar_combustible, el estado del motor) quedan ocultos gracias a atributos/métodos privados (__nombre).

Así, la clase muestra solo lo necesario y oculta lo irrelevante: esa es la abstracción.